I/O Multiplexing with select() - Event-Driven Architecture

Theory, Edge Cases & Code Examples

- ✓ The Blocking Problem
- ✓ What is select()?
- ✓ The File Descriptor Set
- ✓ Step by Step
- ✓ What "Ready" Means for Each Set

→ Swipe to learn →

I/O Multiplexing with select() - Event-Driven Architecture

Theory

📌 The Blocking Problem

In Topics 1 and 2, we learned to handle multiple clients using processes or threads. But both approaches have limitations:

Process/Thread per Client:

```
// Thread-based approach from Topic 2
while (true) {
    int client_fd = accept(server_fd, NULL, NULL);  // ❌ Blocks waiting for client
    std::thread t(handle_client, client_fd);
    t.detach();
}
```

Problems:
1. **Resource cost**: Each client needs a thread (~8KB stack + overhead)
2. **Context switching**: 1000 threads = expensive kernel scheduling
3. **Blocked even when idle**: Thread waits in `recv()` even if no data
4. **Scaling limits**: Most systems limit ~10,000-30,000 threads

What if we could monitor multiple connections with ONE thread?

This is I/O Multiplexing - the ability to monitor multiple file descriptors (sockets) and only process those that are ready, all from a single thread.

Real-world analogy:
- **Threads**: Hiring one waiter per restaurant table (expensive, limited)
- **I/O Multiplexing**: One waiter watches all tables, only serves when customer signals (efficient, scalable)

I/O Multiplexing with select() - Event-Driven Architecture

Theory

📌 What is select()?

`select()` is a POSIX system call that blocks until one or more file descriptors become "ready" for I/O operations without blocking.

Function signature:

```
#include <sys/select.h>

int select(int nfds,                      // Highest FD + 1
           fd_set *readfds,               // Monitor for reading
           fd_set *writefds,              // Monitor for writing
           fd_set *exceptfds,             // Monitor for exceptions
           struct timeval *timeout);      // NULL = block forever
```

Returns:
- Positive number: Count of ready file descriptors
- 0: Timeout expired (no FDs ready)
- -1: Error (check errno)

Key insight: `select()` tells you WHICH file descriptors are ready, not what data they contain.

I/O Multiplexing with select() - Event-Driven Architecture

Theory

📌 fd_set - The File Descriptor Set

`fd_set` is a data structure (typically a bitmask) that represents a set of file descriptors.

Manipulation macros:

```
fd_set read_fds;

FD_ZERO(&read_fds);         // Clear all bits (empty set)
FD_SET(fd, &read_fds);      // Add fd to set
FD_CLR(fd, &read_fds);      // Remove fd from set
FD_ISSET(fd, &read_fds);    // Test if fd is in set (returns true/false)
```

Important: `fd_set` is LIMITED to 1024 file descriptors on most systems (FD_SETSIZE).

Internal representation (simplified):

```
// Conceptual implementation (actual is more complex)
typedef struct {
    uint32_t bits[32];  // 32 * 32 = 1024 bits
} fd_set;

// FD_SET(5, &fds) sets bit 5 to 1
// FD_ISSET(5, &fds) checks if bit 5 is 1
```

I/O Multiplexing with select() - Event-Driven Architecture

Theory

📌 How select() Works - Step by Step

Basic flow:

```
fd_set read_fds;
FD_ZERO(&read_fds);
FD_SET(server_fd, &read_fds);      // Monitor listening socket
FD_SET(client1_fd, &read_fds);     // Monitor client 1
FD_SET(client2_fd, &read_fds);     // Monitor client 2

int max_fd = std::max({server_fd, client1_fd, client2_fd});

// ❌ CRITICAL: select() MODIFIES read_fds
int ready = select(max_fd + 1, &read_fds, NULL, NULL, NULL);

if (ready > 0) {
    // Check which FDs are ready
    if (FD_ISSET(server_fd, &read_fds)) {
        // New client connecting
        int new_client = accept(server_fd, NULL, NULL);
    }

    if (FD_ISSET(client1_fd, &read_fds)) {
        // Client 1 has data to read
        recv(client1_fd, buffer, sizeof(buffer), 0);
    }

    if (FD_ISSET(client2_fd, &read_fds)) {
        // Client 2 has data to read
        recv(client2_fd, buffer, sizeof(buffer), 0);
    }
}
```

Key points:

1. **select() modifies fd_set**: After return, `read_fds` only contains ready FDs
2. **Must rebuild fd_set**: Save original set and restore before each select()
3. **nfds parameter**: Must be (highest FD + 1), not count of FDs
4. **Blocking behavior**: Returns when any FD is ready (or timeout)

I/O Multiplexing with select() - Event-Driven Architecture

Theory

📌 What "Ready" Means for Each Set

readfds (reading):
- **Listening socket**: New connection available (accept() won't block)
- **Connected socket**: Data available to read (recv() won't block)
- **Connected socket**: EOF received (recv() returns 0, client disconnected)

writefds (writing):
- **Socket**: Can write without blocking (send() won't block)
- **Useful when**: Socket send buffer was full, now has space

exceptfds (exceptions):
- **Socket**: Out-of-band data received (TCP urgent data)
- **Rarely used**: Most applications ignore this

I/O Multiplexing with select() - Event-Driven Architecture

Theory

📌 Timeout Handling

The `timeout` parameter controls how long `select()` waits:

```
struct timeval timeout;
timeout.tv_sec = 5;       // 5 seconds
timeout.tv_usec = 500000; // + 500,000 microseconds (0.5 sec) = 5.5 sec total

int ready = select(max_fd + 1, &read_fds, NULL, NULL, &timeout);

if (ready == 0) {
    // Timeout expired - no activity for 5.5 seconds
    std::cout << "No activity, sending heartbeat...\n";
}
```

Three timeout modes:

1. **NULL**: Block forever until FD ready
2. **{0, 0}**: Non-blocking poll (return immediately)
3. **{sec, usec}**: Block up to specified time

Real-world use cases:

- Heartbeat timers: Detect idle connections
- Periodic tasks: Check health, send pings
- Connection timeouts: Close inactive clients

I/O Multiplexing with select() - Event-Driven Architecture

Theory

📌 The Master Set Pattern (Essential)

Since `select()` modifies `fd_set`, we must maintain a "master" set:

```
fd_set master_read_fds;   // Original set
fd_set read_fds;          // Working copy for select()
int max_fd = server_fd;

FD_ZERO(&master_read_fds);
FD_SET(server_fd, &master_read_fds);

while (true) {
    read_fds = master_read_fds;  // ✅ Copy master to working set

    int ready = select(max_fd + 1, &read_fds, NULL, NULL, NULL);

    // Process ready FDs (read_fds is now modified)

    // Next iteration: restore from master_read_fds
}
```

Why this works:
- `master_read_fds`: Permanent record of all sockets to monitor
- `read_fds`: Temporary working copy for each select() call
- After select() returns, `read_fds` contains only ready FDs
- Next iteration: Copy master → working copy → select() again

---

I/O Multiplexing with select() - Event-Driven Architecture

Theory

📌 Handling New Connections

When server socket is ready, it means a client is trying to connect:

```
if (FD_ISSET(server_fd, &read_fds)) {
    struct sockaddr_in client_addr;
    socklen_t addr_len = sizeof(client_addr);

    int new_client = accept(server_fd, (struct sockaddr*)&client_addr, &addr_len);

    if (new_client < 0) {
        perror("accept");
        continue;
    }

    // ✅ Add new client to master set
    FD_SET(new_client, &master_read_fds);

    // ✅ Update max_fd if needed
    if (new_client > max_fd) {
        max_fd = new_client;
    }

    std::cout << "New client " << new_client << " connected\n";
}
```

I/O Multiplexing with select() - Event-Driven Architecture

Theory

📌 Handling Client Disconnection

When `recv()` returns 0, client has closed connection:

```
char buffer[1024];
int n = recv(client_fd, buffer, sizeof(buffer), 0);

if (n == 0) {
    // Client disconnected gracefully
    std::cout << "Client " << client_fd << " disconnected\n";

    // ✅ Remove from master set
    FD_CLR(client_fd, &master_read_fds);

    // ✅ Close socket
    close(client_fd);

    // Note: max_fd doesn't need immediate update (inefficiency but safe)
    // Can recalculate if needed
} else if (n < 0) {
    // Error reading
    perror("recv");
    FD_CLR(client_fd, &master_read_fds);
    close(client_fd);
} else {
    // Data received
    buffer[n] = '\0';
    std::cout << "Received from " << client_fd << ": " << buffer << "\n";
}
```

I/O Multiplexing with select() - Event-Driven Architecture

Theory

📌 select() vs. Threads/Processes - Comparison

Aspect	select()	Threads	Processes (fork)
Model	Event-driven, single-threaded	Multi-threaded	Multi-process
Memory	Low (~few KB)	Medium (~8KB per thread)	High (~10MB per process)
Max Clients	1024 (FD_SETSIZE limit)	~10,000 (thread limit)	~1,000 (process limit)
Complexity	State machine (complex)	Linear flow (simple)	Linear flow (simple)
Context Switch	None (single thread)	Kernel scheduling	Kernel scheduling
Scalability	Good for <1000 clients	Moderate	Poor
Use Case	High concurrency, idle connections	Moderate concurrency, CPU-bound	Security isolation

When to use select():
- ✅ Many concurrent connections (100-1000)
- ✅ Most connections are idle (chat servers, multiplayer games)
- ✅ Single-core CPU or want to minimize context switches
- ❌ Complex business logic per client (state machine becomes hard)
- ❌ Need to scale beyond 1024 connections (use epoll/kqueue instead)

I/O Multiplexing with select() - Event-Driven Architecture

Theory

📌 Limitations of select()

1. **FD_SETSIZE limit (1024)**:
2. Cannot monitor more than 1024 file descriptors
3. Recompiling with larger FD_SETSIZE is not portable
4. **O(n) complexity**:
5. Must iterate through all FDs to check FD_ISSET()
6. Kernel must copy entire fd_set on each call
7. **fd_set modification**:
8. Must rebuild fd_set before each call
9. Easy to forget and cause bugs
0. **max_fd + 1 parameter**:
1. Must track highest FD number
2. Closing highest FD doesn't reduce max_fd (inefficiency)
3. **No edge-triggered mode**:
4. Level-triggered only (reports "ready" every time)
5. Can't detect "new data arrived since last check"

Solutions: `poll()` (Topic 4) fixes some issues, `epoll()` (Topic 5) fixes all

---

I/O Multiplexing with select() - Event-Driven Architecture

Theory

📌 Real-World Relevance

Where select() is still used:
1. **Embedded systems**: Simple, well-tested, portable
2. **Small-scale servers**: <100 clients, simplicity over performance
3. **Cross-platform code**: Works on all UNIX-like systems (including old ones)
4. **Educational purposes**: Teaches event-driven programming concepts

Where select() should NOT be used:
1. High-performance servers (>1000 clients) - Use epoll/kqueue
2. Modern Linux-only applications - Use epoll directly
3. Applications needing >1024 connections - Use epoll/poll

Autonomous vehicle context:
- **Cloud gateway**: Single process handles 100-1000 vehicle connections using select()
- **Vehicle ECU communication**: Monitor multiple CAN bus interfaces with one thread
- **Sensor fusion**: Collect data from multiple sensors using select() instead of threads

---

I/O Multiplexing with select() - Event-Driven Architecture

Edge Cases

📌 1. FD_SETSIZE Overflow (1024 Limit) (1/2)

Problem: Adding more than 1024 file descriptors causes undefined behavior.

```
// ❌ Dangerous: No check before FD_SET
int new_client = accept(server_fd, NULL, NULL);
FD_SET(new_client, &master_read_fds);  // If new_client >= 1024, corruption!
```

Symptoms:
- Segmentation fault
- Memory corruption
- Random crashes

Why it happens:
- FD numbers are not reused immediately
- If you've opened/closed many files, FD numbers increment
- FD 1024+ exceeds fd_set bounds

Solution: Check FD number before adding to set

```
int new_client = accept(server_fd, NULL, NULL);

if (new_client >= FD_SETSIZE) {
    // ✅ Reject connection gracefully
    const char* msg = "HTTP/1.1 503 Service Unavailable\r\nContent-Length: 0\r\n\r\n";
    send(new_client, msg, strlen(msg), 0);
    close(new_client);
    std::cerr << "Connection limit reached (FD_SETSIZE=" << FD_SETSIZE << ")\n";
} else {
    FD_SET(new_client, &master_read_fds);
    if (new_client > max_fd) {
        max_fd = new_client;
    }
}
```

Production pattern: Track active connection count

I/O Multiplexing with select() - Event-Driven Architecture

Edge Cases

📌 1. FD_SETSIZE Overflow (1024 Limit) (2/2)

```
std::atomic<int> active_clients{0};
const int MAX_CLIENTS = 900;  // Leave buffer below FD_SETSIZE

if (active_clients >= MAX_CLIENTS) {
    const char* msg = "503 Server Full\r\n";
    send(new_client, msg, strlen(msg), 0);
    close(new_client);
} else {
    FD_SET(new_client, &master_read_fds);
    active_clients++;
}
```

I/O Multiplexing with select() - Event-Driven Architecture

Edge Cases

📌 2. Forgetting to Copy Master Set

Problem: Directly using master set causes it to be modified by select().

```
fd_set master_fds;
FD_SET(server_fd, &master_fds);
FD_SET(client1_fd, &master_fds);

while (true) {
    // ❌ Passing master set directly
    int ready = select(max_fd + 1, &master_fds, NULL, NULL, NULL);

    // Now master_fds only contains ready FDs
    // Next iteration: Only monitors FDs that were ready last time!
}
```

Symptoms:
- Connections become unresponsive
- Only recently active clients are monitored
- Intermittent behavior (depends on timing)

Why it happens:
- `select()` modifies the fd_set in place
- Removes all non-ready FDs from the set
- Next call only monitors FDs that were ready in previous call

Solution: Always copy master set before select()

```
fd_set master_fds, read_fds;
FD_ZERO(&master_fds);
FD_SET(server_fd, &master_fds);

while (true) {
    read_fds = master_fds;  // ✅ Copy master to working set

    int ready = select(max_fd + 1, &read_fds, NULL, NULL, NULL);

    // Process read_fds (modified by select)
    // Next iteration: read_fds = master_fds (restore)
}
```

I/O Multiplexing with select() - Event-Driven Architecture

Edge Cases

📌 3. Partial Send and Write Blocking (1/2)

Problem: `send()` can write fewer bytes than requested when socket buffer is full.

```
// ❌ Assumes send() writes all bytes
const char* message = "Large message with 10000 bytes...";
send(client_fd, message, strlen(message), 0);  // May only send 2000 bytes!
```

Why it happens:
- Socket send buffer has limited size (~64KB default)
- If buffer is full, `send()` writes what it can and returns
- Remaining bytes must be sent later

Solution 1: Loop until all bytes sent

```
int send_all(int fd, const char* data, int len) {
    int total_sent = 0;

    while (total_sent < len) {
        int n = send(fd, data + total_sent, len - total_sent, 0);

        if (n < 0) {
            if (errno == EAGAIN || errno == EWOULDBLOCK) {
                // Socket buffer full, try again later
                break;
            }
            perror("send");
            return -1;
        }

        total_sent += n;
    }

    return total_sent;
}
```

I/O Multiplexing with select() - Event-Driven Architecture

Edge Cases

📌 3. Partial Send and Write Blocking (2/2)

Solution 2: Monitor writefds and queue partial sends

```
std::map<int, std::string> write_buffers;  // FD → pending data

// When sending large message:
int n = send(client_fd, data, len, 0);
if (n < len) {
    // Partial send - queue remainder
    write_buffers[client_fd] = std::string(data + n, len - n);
    FD_SET(client_fd, &master_write_fds);  // Monitor for writing
}

// In select() loop:
fd_set write_fds = master_write_fds;
select(max_fd + 1, &read_fds, &write_fds, NULL, NULL);

if (FD_ISSET(client_fd, &write_fds)) {
    // Socket ready for writing - send queued data
    std::string& buffer = write_buffers[client_fd];
    int n = send(client_fd, buffer.c_str(), buffer.size(), 0);

    if (n == buffer.size()) {
        // All data sent
        write_buffers.erase(client_fd);
        FD_CLR(client_fd, &master_write_fds);
    } else {
        // Still partial - update buffer
        buffer.erase(0, n);
    }
}
```

I/O Multiplexing with select() - Event-Driven Architecture

Edge Cases

📌 4. Signal Interruption (EINTR)

Problem: `select()` can be interrupted by signals (e.g., SIGCHLD, SIGALRM).

```
// ❌ No EINTR handling
int ready = select(max_fd + 1, &read_fds, NULL, NULL, NULL);

// If SIGCHLD arrives, select() returns -1 with errno=EINTR
// Code may treat this as fatal error and exit
```

Why it happens:
- Signals interrupt blocking system calls
- `select()` returns -1 with `errno = EINTR`
- This is NOT an error - it's normal behavior

Solution: Retry on EINTR

```
int ready;
do {
    read_fds = master_fds;
    ready = select(max_fd + 1, &read_fds, NULL, NULL, NULL);
} while (ready < 0 && errno == EINTR);

if (ready < 0) {
    perror("select");
    break;
}
```

Better solution: Use SA_RESTART flag (auto-restart interrupted syscalls)

```
// When installing signal handler:
struct sigaction sa;
sa.sa_handler = sigchld_handler;
sigemptyset(&sa.sa_mask);
sa.sa_flags = SA_RESTART;  // ✅ Auto-restart select() after signal
sigaction(SIGCHLD, &sa, NULL);

// Now select() automatically restarts after SIGCHLD
int ready = select(max_fd + 1, &read_fds, NULL, NULL, NULL);
```

I/O Multiplexing with select() - Event-Driven Architecture

Edge Cases

📌 5. Disconnect Detection with Zero-Byte Read (1/2)

Problem: Not checking for zero-byte read causes infinite loop.

```
// ❌ Doesn't check for disconnection
if (FD_ISSET(client_fd, &read_fds)) {
    char buffer[1024];
    int n = recv(client_fd, buffer, sizeof(buffer), 0);

    // If n == 0 (client disconnected), FD stays in master_fds
    // Next iteration: select() reports FD ready (EOF)
    // recv() returns 0 again, infinite loop!

    std::cout << "Received: " << std::string(buffer, n) << "\n";
}
```

Symptoms:
- select() returns immediately every iteration
- CPU usage spikes to 100%
- Busy loop on disconnected socket

Why it happens:
- When client closes connection, socket enters EOF state
- EOF is "readable" (recv() returns 0 without blocking)
- If not removed from master_fds, select() reports it ready forever

Solution: Always check for zero-byte read and remove FD

```
if (FD_ISSET(client_fd, &read_fds)) {
    char buffer[1024];
    int n = recv(client_fd, buffer, sizeof(buffer), 0);

    if (n == 0) {
        // ✅ Client disconnected
        std::cout << "Client " << client_fd << " disconnected\n";
        FD_CLR(client_fd, &master_fds);
        close(client_fd);
        active_clients--;
    } else if (n < 0) {
        // ✅ Error
// ...continues on next slide
```

I/O Multiplexing with select() - Event-Driven Architecture

Edge Cases

📌 5. Disconnect Detection with Zero-Byte Read (2/2)

[Code continues across slides]

```
// ...continued from previous slide
        perror("recv");
        FD_CLR(client_fd, &master_fds);
        close(client_fd);
        active_clients--;
    } else {
        // ✅ Data received
        buffer[n] = '\0';
        std::cout << "Received: " << buffer << "\n";
    }
}
```

I/O Multiplexing with select() - Event-Driven Architecture

Code Examples

📌 1. Basic select() Echo Server (1/4)

Purpose: Minimal echo server using select() to handle multiple clients.

Concepts: Master set pattern, accept handling, echo logic.

Compilation:

Testing:

Output:

```
#include <iostream>
#include <cstring>
#include <unistd.h>
#include <sys/socket.h>
#include <sys/select.h>
#include <netinet/in.h>
#include <arpa/inet.h>

const int PORT = 8080;
const int BUFFER_SIZE = 1024;

int main() {
    // Create listening socket
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (server_fd < 0) {
        perror("socket");
        return 1;
    }

    // Allow address reuse
    int opt = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    // Bind to port
    struct sockaddr_in server_addr;
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(PORT);

    if (bind(server_fd, (struct sockaddr*)&server_addr, sizeof(server_addr)) < 0) {
        perror("bind");
// ...continues on next slide
```

I/O Multiplexing with select() - Event-Driven Architecture

Code Examples

📌 1. Basic select() Echo Server (2/4)

```
// ...continued from previous slide
        close(server_fd);
        return 1;
    }

    // Listen
    if (listen(server_fd, 10) < 0) {
        perror("listen");
        close(server_fd);
        return 1;
    }

    std::cout << "Server listening on port " << PORT << "\n";

    // Initialize master fd_set
    fd_set master_fds;
    FD_ZERO(&master_fds);
    FD_SET(server_fd, &master_fds);
    int max_fd = server_fd;

    // Main loop
    while (true) {
        // Copy master to working set
        fd_set read_fds = master_fds;

        // Wait for activity
        int ready = select(max_fd + 1, &read_fds, NULL, NULL, NULL);

        if (ready < 0) {
            perror("select");
            break;
        }

        // Check each file descriptor
        for (int fd = 0; fd <= max_fd; fd++) {
            if (!FD_ISSET(fd, &read_fds)) {
                continue;  // Not ready
            }

            if (fd == server_fd) {
                // New connection on listening socket
// ...continues on next slide
```

I/O Multiplexing with select() - Event-Driven Architecture

Code Examples

📌 1. Basic select() Echo Server (3/4)

```
// ...continued from previous slide
                struct sockaddr_in client_addr;
                socklen_t addr_len = sizeof(client_addr);
                int new_client = accept(server_fd, (struct sockaddr*)&client_addr, &addr_len);

                if (new_client < 0) {
                    perror("accept");
                    continue;
                }

                // Add to master set
                FD_SET(new_client, &master_fds);
                if (new_client > max_fd) {
                    max_fd = new_client;
                }

                std::cout << "New client " << new_client << " connected from "
                          << inet_ntoa(client_addr.sin_addr) << "\n";

            } else {
                // Data from existing client
                char buffer[BUFFER_SIZE];
                int n = recv(fd, buffer, sizeof(buffer), 0);

                if (n <= 0) {
                    // Client disconnected or error
                    if (n == 0) {
                        std::cout << "Client " << fd << " disconnected\n";
                    } else {
                        perror("recv");
                    }

                    close(fd);
                    FD_CLR(fd, &master_fds);

                } else {
                    // Echo back to client
                    buffer[n] = '\0';
                    std::cout << "Client " << fd << ": " << buffer;

                    send(fd, buffer, n, 0);
// ...continues on next slide
```

I/O Multiplexing with select() - Event-Driven Architecture

Code Examples

📌 1. Basic select() Echo Server (4/4)

[Code continues across slides]

```
// ...continued from previous slide
                }
            }
        }
    }

    close(server_fd);
    return 0;
}
```

```
['**Testing**:', '**Output**:']
```

I/O Multiplexing with select() - Event-Driven Architecture

Code Examples

📌 2. select() with Client Management (1/4)

Purpose: Track connected clients with metadata and broadcast messages.

Concepts: Client registry, broadcast to all, FD_SETSIZE checking.

Testing:

```
#include <iostream>
#include <string>
#include <map>
#include <cstring>
#include <unistd.h>
#include <sys/socket.h>
#include <sys/select.h>
#include <netinet/in.h>
#include <arpa/inet.h>

const int PORT = 8080;
const int BUFFER_SIZE = 1024;

struct Client {
    std::string address;
    int port;
    time_t connected_at;
};

std::map<int, Client> clients;  // FD → Client info

void broadcast(const std::string& message, int sender_fd) {
    std::string msg = "Client " + std::to_string(sender_fd) + ": " + message;

    for (const auto& [fd, client] : clients) {
        if (fd != sender_fd) {
            send(fd, msg.c_str(), msg.size(), 0);
        }
    }
}

int main() {
    // Create and bind server socket (same as Example 1)
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);
// ...continues on next slide
```

I/O Multiplexing with select() - Event-Driven Architecture

Code Examples

📌 2. select() with Client Management (2/4)

```
// ...continued from previous slide
    int opt = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    struct sockaddr_in server_addr;
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(PORT);

    bind(server_fd, (struct sockaddr*)&server_addr, sizeof(server_addr));
    listen(server_fd, 10);

    std::cout << "Chat server listening on port " << PORT << "\n";

    fd_set master_fds;
    FD_ZERO(&master_fds);
    FD_SET(server_fd, &master_fds);
    int max_fd = server_fd;

    while (true) {
        fd_set read_fds = master_fds;
        int ready = select(max_fd + 1, &read_fds, NULL, NULL, NULL);

        if (ready < 0) {
            perror("select");
            break;
        }

        for (int fd = 0; fd <= max_fd; fd++) {
            if (!FD_ISSET(fd, &read_fds)) continue;

            if (fd == server_fd) {
                // New connection
                struct sockaddr_in client_addr;
                socklen_t addr_len = sizeof(client_addr);
                int new_client = accept(server_fd, (struct sockaddr*)&client_addr, &addr_len);

                if (new_client < 0) {
                    perror("accept");
                    continue;
                }
// ...continues on next slide
```

I/O Multiplexing with select() - Event-Driven Architecture

Code Examples

📌 2. select() with Client Management (3/4)

```
// ...continued from previous slide

                // ✅ Check FD_SETSIZE limit
                if (new_client >= FD_SETSIZE) {
                    const char* msg = "503 Server Full\r\n";
                    send(new_client, msg, strlen(msg), 0);
                    close(new_client);
                    std::cerr << "Rejected connection (FD_SETSIZE limit)\n";
                    continue;
                }

                // Add to master set and client registry
                FD_SET(new_client, &master_fds);
                if (new_client > max_fd) {
                    max_fd = new_client;
                }

                Client client_info;
                client_info.address = inet_ntoa(client_addr.sin_addr);
                client_info.port = ntohs(client_addr.sin_port);
                client_info.connected_at = time(NULL);
                clients[new_client] = client_info;

                std::cout << "Client " << new_client << " connected from "
                          << client_info.address << ":" << client_info.port
                          << " (Total: " << clients.size() << ")\n";

                // Welcome message
                std::string welcome = "Welcome! You are client " + std::to_string(new_client) + "\n";
                send(new_client, welcome.c_str(), welcome.size(), 0);

            } else {
                // Data from client
                char buffer[BUFFER_SIZE];
                int n = recv(fd, buffer, sizeof(buffer), 0);

                if (n <= 0) {
                    // Disconnected
                    if (n == 0) {
                        std::cout << "Client " << fd << " disconnected\n";
                    } else {
// ...continues on next slide
```

I/O Multiplexing with select() - Event-Driven Architecture

Code Examples

📌 2. select() with Client Management (4/4)

[Code continues across slides]

```
// ...continued from previous slide
                        perror("recv");
                    }

                    close(fd);
                    FD_CLR(fd, &master_fds);
                    clients.erase(fd);

                } else {
                    // Broadcast to all other clients
                    buffer[n] = '\0';
                    std::string message(buffer);

                    // Remove trailing newline
                    if (!message.empty() && message.back() == '\n') {
                        message.pop_back();
                    }

                    std::cout << "Client " << fd << ": " << message << "\n";
                    broadcast(message + "\n", fd);
                }
            }
        }
    }

    close(server_fd);
    return 0;
}
```

I/O Multiplexing with select() - Event-Driven Architecture

Code Examples

📌 3. Handling Partial Sends with Write Monitoring (1/4)

Purpose: Demonstrate proper handling of partial sends using writefds.

Concepts: Write buffer queue, monitoring writefds, partial send handling.

Key points:
- Uses both `readfds` and `writefds` in select()
- Queues partial sends in `write_buffers` map
- Only monitors writefds when there's pending data
- Removes from writefds when queue is empty

```
#include <iostream>
#include <string>
#include <map>
#include <queue>
#include <cstring>
#include <unistd.h>
#include <sys/socket.h>
#include <sys/select.h>
#include <netinet/in.h>

const int PORT = 8080;
const int BUFFER_SIZE = 1024;

// Write buffers for each client (queue of pending data)
std::map<int, std::queue<std::string>> write_buffers;

int send_data(int fd, const std::string& data) {
    int n = send(fd, data.c_str(), data.size(), 0);

    if (n < 0) {
        if (errno == EAGAIN || errno == EWOULDBLOCK) {
            return 0;  // Would block, queue entire message
        }
        perror("send");
        return -1;
    }

    return n;
// ...continues on next slide
```

I/O Multiplexing with select() - Event-Driven Architecture

Code Examples

📌 3. Handling Partial Sends with Write Monitoring (2/4)

```
// ...continued from previous slide
}

void queue_write(int fd, const std::string& data) {
    if (write_buffers.find(fd) == write_buffers.end()) {
        write_buffers[fd] = std::queue<std::string>();
    }
    write_buffers[fd].push(data);
}

int main() {
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);
    int opt = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    struct sockaddr_in server_addr;
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(PORT);

    bind(server_fd, (struct sockaddr*)&server_addr, sizeof(server_addr));
    listen(server_fd, 10);

    std::cout << "Server with partial send handling listening on port " << PORT << "\n";

    fd_set master_read_fds, master_write_fds;
    FD_ZERO(&master_read_fds);
    FD_ZERO(&master_write_fds);
    FD_SET(server_fd, &master_read_fds);
    int max_fd = server_fd;

    while (true) {
        fd_set read_fds = master_read_fds;
        fd_set write_fds = master_write_fds;

        int ready = select(max_fd + 1, &read_fds, &write_fds, NULL, NULL);

        if (ready < 0) {
            perror("select");
            break;
        }
// ...continues on next slide
```

I/O Multiplexing with select() - Event-Driven Architecture

Code Examples

📌 3. Handling Partial Sends with Write Monitoring (3/4)

```
// ...continued from previous slide

        // Handle readable FDs
        for (int fd = 0; fd <= max_fd; fd++) {
            if (FD_ISSET(fd, &read_fds)) {
                if (fd == server_fd) {
                    // Accept new connection
                    int new_client = accept(server_fd, NULL, NULL);
                    if (new_client >= 0 && new_client < FD_SETSIZE) {
                        FD_SET(new_client, &master_read_fds);
                        if (new_client > max_fd) max_fd = new_client;
                        std::cout << "Client " << new_client << " connected\n";
                    }
                } else {
                    // Receive data
                    char buffer[BUFFER_SIZE];
                    int n = recv(fd, buffer, sizeof(buffer), 0);

                    if (n <= 0) {
                        std::cout << "Client " << fd << " disconnected\n";
                        close(fd);
                        FD_CLR(fd, &master_read_fds);
                        FD_CLR(fd, &master_write_fds);
                        write_buffers.erase(fd);
                    } else {
                        // Echo back (may be partial send)
                        std::string response(buffer, n);
                        int sent = send_data(fd, response);

                        if (sent < n) {
                            // Partial send - queue remainder
                            std::string remainder = response.substr(sent);
                            queue_write(fd, remainder);
                            FD_SET(fd, &master_write_fds);  // Monitor for writing
                            std::cout << "Partial send on " << fd << ": " << sent << "/" << n << " bytes\n";
                        }
                    }
                }
            }
        }

// ...continues on next slide
```

I/O Multiplexing with select() - Event-Driven Architecture

Code Examples

📌 3. Handling Partial Sends with Write Monitoring (4/4)

[Code continues across slides]

```
// ...continued from previous slide
        // Handle writable FDs (process queued writes)
        for (int fd = 0; fd <= max_fd; fd++) {
            if (FD_ISSET(fd, &write_fds)) {
                if (write_buffers.find(fd) != write_buffers.end() && !write_buffers[fd].empty()) {
                    std::string& data = write_buffers[fd].front();
                    int sent = send_data(fd, data);

                    if (sent < 0) {
                        // Error - close connection
                        close(fd);
                        FD_CLR(fd, &master_read_fds);
                        FD_CLR(fd, &master_write_fds);
                        write_buffers.erase(fd);
                    } else if (sent == data.size()) {
                        // Complete send - remove from queue
                        write_buffers[fd].pop();

                        if (write_buffers[fd].empty()) {
                            // No more data to send
                            FD_CLR(fd, &master_write_fds);
                            std::cout << "Write queue empty for " << fd << "\n";
                        }
                    } else {
                        // Still partial - update buffer
                        data = data.substr(sent);
                        std::cout << "Still partial send on " << fd << ": " << sent << " bytes\n";
                    }
                }
            }
        }
    }

    close(server_fd);
    return 0;
}
```

I/O Multiplexing with select() - Event-Driven Architecture

Code Examples

📌 4. Timeout-Based Heartbeat (1/4)

Purpose: Use select() timeout to implement periodic tasks (heartbeat, idle detection).

Concepts: Timeout handling, periodic tasks, idle client detection.

Output:

```
#include <iostream>
#include <map>
#include <cstring>
#include <ctime>
#include <unistd.h>
#include <sys/socket.h>
#include <sys/select.h>
#include <netinet/in.h>

const int PORT = 8080;
const int HEARTBEAT_INTERVAL = 5;  // seconds
const int IDLE_TIMEOUT = 30;       // seconds

struct ClientInfo {
    time_t last_activity;
};

std::map<int, ClientInfo> clients;

void check_idle_clients(fd_set& master_fds) {
    time_t now = time(NULL);
    std::vector<int> to_remove;

    for (auto& [fd, info] : clients) {
        if (now - info.last_activity > IDLE_TIMEOUT) {
            std::cout << "Client " << fd << " idle for " << (now - info.last_activity)
                      << " seconds - disconnecting\n";

            const char* msg = "Disconnected due to inactivity\n";
            send(fd, msg, strlen(msg), 0);

            close(fd);
            FD_CLR(fd, &master_fds);
            to_remove.push_back(fd);
// ...continues on next slide
```

I/O Multiplexing with select() - Event-Driven Architecture

Code Examples

📌 4. Timeout-Based Heartbeat (2/4)

```
// ...continued from previous slide
        }
    }

    for (int fd : to_remove) {
        clients.erase(fd);
    }
}

void send_heartbeats() {
    const char* ping = "PING\n";
    for (const auto& [fd, info] : clients) {
        send(fd, ping, strlen(ping), 0);
    }
    std::cout << "Sent heartbeat to " << clients.size() << " clients\n";
}

int main() {
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);
    int opt = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    struct sockaddr_in server_addr;
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(PORT);

    bind(server_fd, (struct sockaddr*)&server_addr, sizeof(server_addr));
    listen(server_fd, 10);

    std::cout << "Server with heartbeat/timeout on port " << PORT << "\n";

    fd_set master_fds;
    FD_ZERO(&master_fds);
    FD_SET(server_fd, &master_fds);
    int max_fd = server_fd;

    time_t last_heartbeat = time(NULL);

    while (true) {
        fd_set read_fds = master_fds;
// ...continues on next slide
```

I/O Multiplexing with select() - Event-Driven Architecture

Code Examples

📌 4. Timeout-Based Heartbeat (3/4)

```
// ...continued from previous slide

        // Set timeout for heartbeat interval
        struct timeval timeout;
        timeout.tv_sec = HEARTBEAT_INTERVAL;
        timeout.tv_usec = 0;

        int ready = select(max_fd + 1, &read_fds, NULL, NULL, &timeout);

        if (ready < 0) {
            perror("select");
            break;
        } else if (ready == 0) {
            // Timeout - no activity
            std::cout << "Timeout: No activity for " << HEARTBEAT_INTERVAL << " seconds\n";

            // Send heartbeats
            send_heartbeats();

            // Check for idle clients
            check_idle_clients(master_fds);

            last_heartbeat = time(NULL);
            continue;
        }

        // Activity detected
        for (int fd = 0; fd <= max_fd; fd++) {
            if (!FD_ISSET(fd, &read_fds)) continue;

            if (fd == server_fd) {
                // New connection
                int new_client = accept(server_fd, NULL, NULL);
                if (new_client >= 0 && new_client < FD_SETSIZE) {
                    FD_SET(new_client, &master_fds);
                    if (new_client > max_fd) max_fd = new_client;

                    ClientInfo info;
                    info.last_activity = time(NULL);
                    clients[new_client] = info;

// ...continues on next slide
```

I/O Multiplexing with select() - Event-Driven Architecture

Code Examples

📌 4. Timeout-Based Heartbeat (4/4)

[Code continues across slides]

```
// ...continued from previous slide
                    std::cout << "Client " << new_client << " connected\n";
                }
            } else {
                // Data from client
                char buffer[1024];
                int n = recv(fd, buffer, sizeof(buffer), 0);

                if (n <= 0) {
                    std::cout << "Client " << fd << " disconnected\n";
                    close(fd);
                    FD_CLR(fd, &master_fds);
                    clients.erase(fd);
                } else {
                    // Update last activity time
                    clients[fd].last_activity = time(NULL);

                    buffer[n] = '\0';
                    std::cout << "Client " << fd << ": " << buffer;
                    send(fd, buffer, n, 0);
                }
            }
        }
    }

    close(server_fd);
    return 0;
}
```

```
['Server with heartbeat/timeout on port 8080\nClient 4 connected\nTimeout: No activity for 5 seconds\nSent heartbeat to
```

I/O Multiplexing with select() - Event-Driven Architecture

Code Examples

📌 5. Non-Blocking Poll (timeout = 0) (1/4)

Purpose: Use select() with zero timeout for non-blocking polling while doing other work.

Concepts: Non-blocking poll, CPU-bound work alongside I/O, main loop design.

Key insights:
- `timeout = {0, 0}` makes select() return immediately
- Returns 0 if no FDs ready, >0 if activity detected
- Allows interleaving I/O handling with CPU work
- **Warning**: Can consume 100% CPU if not rate-limited

Use cases:
- Game servers (render frame while checking network)
- Real-time systems (periodic sensor reading + network)
- Background processing (compress files while accepting uploads)

Production improvement:

```
#include <iostream>
#include <cstring>
#include <unistd.h>
#include <sys/socket.h>
#include <sys/select.h>
#include <netinet/in.h>

const int PORT = 8080;

// Simulate CPU-bound work
int process_batch_job() {
    static int job_counter = 0;
    job_counter++;

    // Simulate computation
    int result = 0;
    for (int i = 0; i < 1000000; i++) {
        result += i;
    }

    if (job_counter % 100 == 0) {
// ...continues on next slide
```

I/O Multiplexing with select() - Event-Driven Architecture

Code Examples

📌 5. Non-Blocking Poll (timeout = 0) (2/4)

```
// ...continued from previous slide
        std::cout << "[JOB] Processed " << job_counter << " batches\n";
    }

    return result;
}

int main() {
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);
    int opt = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    struct sockaddr_in server_addr;
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(PORT);

    bind(server_fd, (struct sockaddr*)&server_addr, sizeof(server_addr));
    listen(server_fd, 10);

    std::cout << "Non-blocking server on port " << PORT << "\n";
    std::cout << "Doing background work while monitoring connections\n";

    fd_set master_fds;
    FD_ZERO(&master_fds);
    FD_SET(server_fd, &master_fds);
    int max_fd = server_fd;

    while (true) {
        fd_set read_fds = master_fds;

        // ✅ Zero timeout = non-blocking poll
        struct timeval timeout;
        timeout.tv_sec = 0;
        timeout.tv_usec = 0;  // Return immediately

        int ready = select(max_fd + 1, &read_fds, NULL, NULL, &timeout);

        if (ready < 0) {
            perror("select");
            break;
// ...continues on next slide
```

I/O Multiplexing with select() - Event-Driven Architecture

Code Examples

📌 5. Non-Blocking Poll (timeout = 0) (3/4)

[Code continues across slides]

```
// ...continued from previous slide
        } else if (ready > 0) {
            // Activity detected - handle I/O
            for (int fd = 0; fd <= max_fd; fd++) {
                if (!FD_ISSET(fd, &read_fds)) continue;

                if (fd == server_fd) {
                    // New connection
                    int new_client = accept(server_fd, NULL, NULL);
                    if (new_client >= 0 && new_client < FD_SETSIZE) {
                        FD_SET(new_client, &master_fds);
                        if (new_client > max_fd) max_fd = new_client;
                        std::cout << "[I/O] Client " << new_client << " connected\n";
                    }
                } else {
                    // Client data
                    char buffer[1024];
                    int n = recv(fd, buffer, sizeof(buffer), 0);

                    if (n <= 0) {
                        std::cout << "[I/O] Client " << fd << " disconnected\n";
                        close(fd);
                        FD_CLR(fd, &master_fds);
                    } else {
                        buffer[n] = '\0';
                        std::cout << "[I/O] Client " << fd << ": " << buffer;
                        send(fd, buffer, n, 0);
                    }
                }
            }
        } else {
            // No activity - do background work
            process_batch_job();
        }

        // Note: This loop runs VERY fast, constantly checking for I/O
        // In production, you'd want some rate limiting or sleep
    }

    close(server_fd);
    return 0;
```

I/O Multiplexing with select() - Event-Driven Architecture

Code Examples

📌 5. Non-Blocking Poll (timeout = 0) (4/4)

```
['// Rate-limit background work\nif (ready == 0) {\n    process_batch_job();\n    usleep(1000);  // Sleep 1ms to avoid
```

I/O Multiplexing with select() - Event-Driven Architecture

Code Examples

📌 6. Production-Grade select() Server (1/9)

Purpose: Comprehensive server combining all best practices from previous examples.

Concepts: FD_SETSIZE checking, partial sends, timeouts, graceful shutdown, error handling.

Compilation:

Testing:

Production features:
- ✅ FD_SETSIZE limit checking
- ✅ Client count limiting (MAX_CLIENTS)
- ✅ Partial send buffering with write queues
- ✅ Graceful shutdown (SIGINT/SIGTERM handling)
- ✅ Idle client detection and cleanup
- ✅ Periodic heartbeat
- ✅ Broadcast messaging
- ✅ Connection metadata tracking
- ✅ Comprehensive error handling
- ✅ Activity timestamp tracking

Real-world deployment considerations:
1. **Logging**: Replace cout with proper logging (syslog/log4cpp)
2. **Configuration**: Load settings from config file
3. **Authentication**: Add client authentication
4. **Rate limiting**: Limit messages per client per second
5. **Protocol**: Implement framing (length-prefix or delimiter)
6. **Metrics**: Track bytes sent/received, connection duration
7. **Health endpoint**: Listen on separate port for monitoring

---

```
#include <iostream>
#include <string>
#include <map>
#include <queue>
#include <vector>
// ...continues on next slide
```

I/O Multiplexing with select() - Event-Driven Architecture

Code Examples

📌 6. Production-Grade select() Server (2/9)

```
// ...continued from previous slide
#include <cstring>
#include <ctime>
#include <csignal>
#include <unistd.h>
#include <sys/socket.h>
#include <sys/select.h>
#include <netinet/in.h>
#include <arpa/inet.h>

const int PORT = 8080;
const int BUFFER_SIZE = 4096;
const int HEARTBEAT_INTERVAL = 10;  // seconds
const int IDLE_TIMEOUT = 60;        // seconds
const int MAX_CLIENTS = 900;        // Leave buffer below FD_SETSIZE

// Global flag for graceful shutdown
volatile sig_atomic_t keep_running = 1;

void signal_handler(int signum) {
    std::cout << "\n[SHUTDOWN] Received signal " << signum << "\n";
    keep_running = 0;
}

struct ClientInfo {
    std::string address;
    int port;
    time_t connected_at;
    time_t last_activity;
    std::queue<std::string> write_queue;  // Buffered writes
};

std::map<int, ClientInfo> clients;

void broadcast_message(const std::string& message, int exclude_fd = -1) {
    for (auto& [fd, client] : clients) {
        if (fd != exclude_fd) {
            client.write_queue.push(message);
        }
    }
}
// ...continues on next slide
```

I/O Multiplexing with select() - Event-Driven Architecture

Code Examples

📌 6. Production-Grade select() Server (3/9)

```
// ...continued from previous slide

void send_to_client(int fd, const std::string& message) {
    if (clients.find(fd) != clients.end()) {
        clients[fd].write_queue.push(message);
    }
}

void cleanup_idle_clients(fd_set& master_read_fds, fd_set& master_write_fds) {
    time_t now = time(NULL);
    std::vector<int> to_remove;

    for (auto& [fd, info] : clients) {
        if (now - info.last_activity > IDLE_TIMEOUT) {
            std::cout << "[TIMEOUT] Client " << fd << " idle for "
                      << (now - info.last_activity) << "s\n";

            const char* msg = "Timeout: Disconnected due to inactivity\n";
            send(fd, msg, strlen(msg), 0);

            close(fd);
            FD_CLR(fd, &master_read_fds);
            FD_CLR(fd, &master_write_fds);
            to_remove.push_back(fd);
        }
    }

    for (int fd : to_remove) {
        clients.erase(fd);
    }

    if (!to_remove.empty()) {
        std::cout << "[INFO] Cleaned up " << to_remove.size()
                  << " idle clients. Active: " << clients.size() << "\n";
    }
}

int main() {
    // Install signal handlers for graceful shutdown
    struct sigaction sa;
    sa.sa_handler = signal_handler;
// ...continues on next slide
```

I/O Multiplexing with select() - Event-Driven Architecture

Code Examples

📌 6. Production-Grade select() Server (4/9)

```
// ...continued from previous slide
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = SA_RESTART;
    sigaction(SIGINT, &sa, NULL);
    sigaction(SIGTERM, &sa, NULL);

    // Create server socket
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (server_fd < 0) {
        perror("socket");
        return 1;
    }

    // Set socket options
    int opt = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    // Bind to address
    struct sockaddr_in server_addr;
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(PORT);

    if (bind(server_fd, (struct sockaddr*)&server_addr, sizeof(server_addr)) < 0) {
        perror("bind");
        close(server_fd);
        return 1;
    }

    // Listen
    if (listen(server_fd, 128) < 0) {
        perror("listen");
        close(server_fd);
        return 1;
    }

    std::cout << "=== Production select() Server ===\n";
    std::cout << "Listening on port " << PORT << "\n";
    std::cout << "Max clients: " << MAX_CLIENTS << "\n";
    std::cout << "Heartbeat: " << HEARTBEAT_INTERVAL << "s\n";
    std::cout << "Idle timeout: " << IDLE_TIMEOUT << "s\n\n";
// ...continues on next slide
```

I/O Multiplexing with select() - Event-Driven Architecture

Code Examples

📌 6. Production-Grade select() Server (5/9)

```
// ...continued from previous slide

    // Initialize fd_sets
    fd_set master_read_fds, master_write_fds;
    FD_ZERO(&master_read_fds);
    FD_ZERO(&master_write_fds);
    FD_SET(server_fd, &master_read_fds);
    int max_fd = server_fd;

    time_t last_heartbeat = time(NULL);

    // Main server loop
    while (keep_running) {
        fd_set read_fds = master_read_fds;
        fd_set write_fds = master_write_fds;

        // Set timeout for heartbeat
        struct timeval timeout;
        timeout.tv_sec = HEARTBEAT_INTERVAL;
        timeout.tv_usec = 0;

        int ready = select(max_fd + 1, &read_fds, &write_fds, NULL, &timeout);

        if (ready < 0) {
            if (errno == EINTR) {
                // Interrupted by signal (SIGINT/SIGTERM)
                std::cout << "[INFO] select() interrupted\n";
                break;
            }
            perror("select");
            break;
        }

        if (ready == 0) {
            // Timeout - periodic maintenance
            std::cout << "[HEARTBEAT] " << clients.size() << " clients connected\n";

            broadcast_message("PING\n");
            cleanup_idle_clients(master_read_fds, master_write_fds);

            last_heartbeat = time(NULL);
// ...continues on next slide
```

I/O Multiplexing with select() - Event-Driven Architecture

Code Examples

📌 6. Production-Grade select() Server (6/9)

```
// ...continued from previous slide
            continue;
        }

        // Handle readable file descriptors
        for (int fd = 0; fd <= max_fd; fd++) {
            if (FD_ISSET(fd, &read_fds)) {
                if (fd == server_fd) {
                    // New connection
                    struct sockaddr_in client_addr;
                    socklen_t addr_len = sizeof(client_addr);
                    int new_client = accept(server_fd, (struct sockaddr*)&client_addr, &addr_len);

                    if (new_client < 0) {
                        perror("accept");
                        continue;
                    }

                    // Check FD_SETSIZE limit
                    if (new_client >= FD_SETSIZE) {
                        std::cerr << "[REJECT] FD " << new_client
                                  << " exceeds FD_SETSIZE (" << FD_SETSIZE << ")\n";
                        const char* msg = "HTTP/1.1 503 Service Unavailable\r\n\r\n";
                        send(new_client, msg, strlen(msg), 0);
                        close(new_client);
                        continue;
                    }

                    // Check client limit
                    if (clients.size() >= MAX_CLIENTS) {
                        std::cerr << "[REJECT] Max clients (" << MAX_CLIENTS << ") reached\n";
                        const char* msg = "Server full. Try again later.\n";
                        send(new_client, msg, strlen(msg), 0);
                        close(new_client);
                        continue;
                    }

                    // Add client
                    FD_SET(new_client, &master_read_fds);
                    if (new_client > max_fd) {
                        max_fd = new_client;
// ...continues on next slide
```

I/O Multiplexing with select() - Event-Driven Architecture

Code Examples

📌 6. Production-Grade select() Server (7/9)

```
// ...continued from previous slide
                    }

                    ClientInfo info;
                    info.address = inet_ntoa(client_addr.sin_addr);
                    info.port = ntohs(client_addr.sin_port);
                    info.connected_at = time(NULL);
                    info.last_activity = time(NULL);
                    clients[new_client] = info;

                    std::cout << "[CONNECT] Client " << new_client << " from "
                              << info.address << ":" << info.port
                              << " (Total: " << clients.size() << ")\n";

                    // Welcome message
                    std::string welcome = "Welcome! You are client " + std::to_string(new_client) + "\n";
                    send_to_client(new_client, welcome);
                    FD_SET(new_client, &master_write_fds);

                } else {
                    // Data from existing client
                    char buffer[BUFFER_SIZE];
                    int n = recv(fd, buffer, sizeof(buffer), 0);

                    if (n <= 0) {
                        // Client disconnected
                        if (n == 0) {
                            std::cout << "[DISCONNECT] Client " << fd << "\n";
                        } else {
                            perror("recv");
                        }

                        close(fd);
                        FD_CLR(fd, &master_read_fds);
                        FD_CLR(fd, &master_write_fds);
                        clients.erase(fd);

                    } else {
                        // Update activity timestamp
                        clients[fd].last_activity = time(NULL);

// ...continues on next slide
```

I/O Multiplexing with select() - Event-Driven Architecture

Code Examples

📌 6. Production-Grade select() Server (8/9)

```
// ...continued from previous slide
                        // Process message
                        buffer[n] = '\0';
                        std::string message(buffer);

                        // Remove trailing newline
                        if (!message.empty() && message.back() == '\n') {
                            message.pop_back();
                        }

                        std::cout << "[MSG] Client " << fd << ": " << message << "\n";

                        // Broadcast to other clients
                        std::string broadcast = "Client " + std::to_string(fd) + ": " + message + "\n";
                        broadcast_message(broadcast, fd);

                        // Mark clients with pending writes
                        for (auto& [client_fd, info] : clients) {
                            if (!info.write_queue.empty()) {
                                FD_SET(client_fd, &master_write_fds);
                            }
                        }
                    }
                }
            }
        }

        // Handle writable file descriptors (queued writes)
        for (int fd = 0; fd <= max_fd; fd++) {
            if (FD_ISSET(fd, &write_fds)) {
                if (clients.find(fd) != clients.end() && !clients[fd].write_queue.empty()) {
                    std::string& data = clients[fd].write_queue.front();

                    int sent = send(fd, data.c_str(), data.size(), 0);

                    if (sent < 0) {
                        // Error - close connection
                        perror("send");
                        close(fd);
                        FD_CLR(fd, &master_read_fds);
                        FD_CLR(fd, &master_write_fds);
// ...continues on next slide
```

I/O Multiplexing with select() - Event-Driven Architecture

Code Examples

📌 6. Production-Grade select() Server (9/9)

[Code continues across slides]

```
// ...continued from previous slide
                        clients.erase(fd);

                    } else if (sent == data.size()) {
                        // Complete send
                        clients[fd].write_queue.pop();

                        if (clients[fd].write_queue.empty()) {
                            // No more pending writes
                            FD_CLR(fd, &master_write_fds);
                        }

                    } else {
                        // Partial send - update buffer
                        data = data.substr(sent);
                    }
                }
            }
        }
    }

    // Graceful shutdown
    std::cout << "\n[SHUTDOWN] Closing all connections...\n";

    for (auto& [fd, info] : clients) {
        const char* msg = "Server shutting down. Goodbye!\n";
        send(fd, msg, strlen(msg), 0);
        close(fd);
    }

    close(server_fd);
    std::cout << "[SHUTDOWN] Server stopped cleanly\n";

    return 0;
}
```

```
['**Testing**:']
```

What You Learned

Key Takeaways

- ✓ The Blocking Problem
- ✓ What is select()?
- ✓ The File Descriptor Set
- ✓ Step by Step

→ Swipe to learn →